End-Semester Examination Study Suite: Core Python Foundations (100 Marks)

Question 1 (20 Marks): Control Statements (`if/elif/else` , `for` , `while` , `break` , `continue` , `pass`)

(a) [5 Marks] Theoretical Syntax & Execution Rules

- **Decision-Making (`if / elif / else`)**: The `if` statement evaluates a boolean condition. Execution follows three golden rules:
  1. Every conditional header must terminate with a **colon (`:`)**.
  2. The code body inside must be **indented** (4 spaces or 1 tab).
  3. In an `if/elif/else` ladder, conditions are evaluated sequentially from top to bottom; as soon as a condition evaluates to `True` , its block executes and the rest of the ladder is skipped.
- **Loops (`for` vs. `while`)**:
  - **`for` Loop**: Used when the exact number of iterations is known in advance or when iterating directly over sequence items.
  - **`while` Loop**: Repeats execution continuously as long as its condition remains `True` . **Infinite Loop Hazard**: Something inside the loop body must update the condition state, or a `break` statement must be encountered.
- **Loop Control Keywords**:
  - **`break`** : Immediately terminates the innermost loop entirely and jumps out to the statement following the loop.
  - **`continue`** : Skips the remainder of the current iteration body and jumps directly to the next iteration cycle.
  - **`pass`** : A null placeholder statement that does nothing; used to satisfy Python's syntactic requirement for an indented block.

(b) [5 Marks] Debugging & Code Tracing Questions

Debugging Task 1: Identify the 2 syntax errors and 1 logical error in the snippet below:

```
# Intended: Print even numbers between 1 and 10
count = 1
while count <= 10
    if count % 2 = 0:
        print(count)
```

```
    count = count  # Bug: Loop condition variable not updated
```

- **Syntax Error 1**: Missing colon (`:`) after `while count <= 10`.
- **Syntax Error 2**: Using assignment operator (`=`) instead of equality operator (`==`) inside `if count % 2 == 0:`.
- **Logical Error**: `count` is not incremented (`count += 1`), resulting in an infinite while loop.

Output Tracing Task 2: State the exact console output of this code:

```
for i in range(1, 6):
    if i == 3:
        continue
    if i == 5:
        break
    print(i, end=" ")
```

- **Answer**: `1 2 4`
- *Explanation*: When `i == 3`, `continue` skips printing `3`. When `i == 5`, `break` terminates the loop completely.

© [5 Marks] Simple Baseline Solution: Number Classification Routine

```
# Simple baseline program processing numbers 1 through 10
numbers = [12, -5, 0, 7, -2, 15, 8]

for num in numbers:
    if num < 0:
        continue  # Skip negative numbers
    elif num == 0:
        print("Zero encountered, stopping processing.")
        break  # Terminate on zero
    else:
        if num % 2 == 0:
            print(f"{num} is Positive Even")
        else:
            print(f"{num} is Positive Odd")
```

(d) [5 Marks] Complex Improvement: Modular OOP Execution Engine with Input Validation and Custom Exception Handling

```
# --- Custom Exception Definition ---
class OutOfRangeError(Exception):
    """Raised when an input integer violates domain limits."""
    def __init__(self, value, min_val, max_val):
        self.value = value
        self.message = f"Value {value} is out of permitted range [{min_val}, {max_va
        super().__init__(self.message)

# --- Modular OOP Number Evaluator ---
class NumberProcessingEngine:
    MIN_LIMIT = 1
    MAX_LIMIT = 100

    def __init__(self, raw_inputs):
        self.raw_inputs = raw_inputs
        self.processed_results = []

    def validate_and_classify(self):
        """Processes number sequence using strict exception checks and loop control.'
        for idx, item in enumerate(self.raw_inputs):
            try:
                # Type Conversion check
                val = int(item)

                # Custom Boundary Check
                if val < NumberProcessingEngine.MIN_LIMIT or val > NumberProcessingEr
                    raise OutOfRangeError(val, NumberProcessingEngine.MIN_LIMIT, Numb

                # Classification Logic
                if val % 15 == 0:
                    category = "FizzBuzz"
                elif val % 3 == 0:
                    category = "Fizz"
                elif val % 5 == 0:
                    category = "Buzz"
                else:
                    category = "Prime/Other"

                self.processed_results.append((val, category))

            except ValueError as ex:
                print(f"[Input Error] Index {idx} ('{item}'): Input must be an intege
                continue  # Skip invalid item and continue loop
```

```
            except OutOfRangeError as ex:
                print(f"[Domain Error] Index {idx}: {ex}")
                continue  # Skip out-of-range item

    def display_summary(self):
        print("\n--- Processing Summary ---")
        for num, cat in self.processed_results:
            print(f"Number: {num:>3} | Classification: {cat}")

if __name__ == "__main__":
    sample_data = ["15", "invalid", "3", "150", "10", "7"]
    engine = NumberProcessingEngine(sample_data)
    engine.validate_and_classify()
    engine.display_summary()
```

Question 2 (20 Marks): Lists, Ranges & Tuples

(a) [5 Marks] Theoretical Principles & Syntax Rules

- **Lists (`[]`)**: Ordered, **mutable** sequences created using square brackets `[]`. Elements can be modified, appended, or removed in-place.
- **Tuples (`()`)**: Ordered, **immutable** sequences created using round parentheses `()`. Operations like `.sort()`, `.reverse()`, and element assignment raise a `TypeError`.
  - **Single-Element Tuple Rule**: A single-item tuple **must** include a trailing comma `t = (42,)`; writing `t = (42)` creates an integer, not a tuple.
  - **Packing & Unpacking**: Tuple elements can be assigned to comma-separated variables in a single line `a, b, c = (1, 2, 3)`.
- **Slicing Syntax `sequence[start:stop:step]`**:
  - `start` is included, `stop` is **excluded**.
  - `lst[::-1]` reverses a sequence.
- **`range(start, stop, step)` Function**: Generates an immutable sequence of numbers on demand. `stop` boundary is strictly exclusive.

(b) [5 Marks] Debugging & Syntax Verification

Debugging Task: Correct the errors in the tuple and list processing snippet:

```
# Task: Attempting tuple modification and single-item declaration
coords = (10, 20)
coords = 15          # Line 3: Error
single_item = (100)     # Line 4: Error
print(type(single_item))
```

- **Line 3 Error**: `TypeError: 'tuple' object does not support item assignment`. Tuples are immutable and elements cannot be overwritten directly.
- **Line 4 Error**: `single_item` resolves to `<class 'int'>`. It must be declared as `single_item = (100,)` with a trailing comma to be recognized as a tuple.

© [5 Marks] Simple Baseline Solution: Sequence Manipulator

```
# Simple baseline manipulating lists, tuples, and ranges
numbers_list = list(range(1, 6))  #

# List Operations
numbers_list.append(6)
numbers_list = 10  # Mutable update

# Tuple Operations & Unpacking
point_tuple = (100, 200, 300)
x, y, z = point_tuple  # Unpacking

print("Modified List:", numbers_list)
print("Reversed List Slice:", numbers_list[::-1])
print(f"Unpacked Tuple Coordinates: x={x}, y={y}, z={z}")
```

(d) [5 Marks] Complex Improvement: Modular OOP Sequence Manager with Type Security and Exception Guards

```
class SequenceProcessingEngine:
    def __init__(self, initial_data):
        self._data_list = []
        self._frozen_record = ()
        self.load_data(initial_data)

    def load_data(self, data):
```

```
        """Validates input sequence and initializes mutable list and immutable tuple.
        if not isinstance(data, (list, tuple, range)):
            raise TypeError("Input must be a valid sequence (list, tuple, or range).'

        self._data_list = list(data)
        self._frozen_record = tuple(data)  # Frozen immutable snapshot

    def safe_slice(self, start, stop, step=1):
        """Extracts sub-sequence with index boundary validation."""
        try:
            length = len(self._data_list)
            if abs(start) > length or abs(stop) > length + 1:
                raise IndexError("Slice indices exceed sequence boundaries.")

            return self._data_list[start:stop:step]
        except TypeError as ex:
            raise TypeError(f"Slice parameters must be integers: {ex}")

    def update_element(self, index, new_value):
        """Updates element in mutable list while guarding immutable snapshot."""
        try:
            self._data_list[index] = new_value
            print(f"[Success] Updated list index {index} to {new_value}.")
        except IndexError:
            raise IndexError(f"Cannot update: Index {index} is out of bounds for list

    def attempt_tuple_mutation(self, index, value):
        """Demonstrates immutability security of internal tuple."""
        try:
            # Intentional immutability check
            self._frozen_record[index] = value
        except TypeError as ex:
            print(f"[Security Protection] Tuple mutation blocked as expected: {ex}")

    def get_summary(self):
        return {
            "mutable_list": self._data_list,
            "immutable_tuple": self._frozen_record,
            "element_count": len(self._data_list)
        }

if __name__ == "__main__":
    try:
        engine = SequenceProcessingEngine(range(5, 50, 10))  # 5, 15, 25, 35, 45
        print("Initial Slice [1:4]:", engine.safe_slice(1, 4))
```

```
        engine.update_element(0, 99)
        engine.attempt_tuple_mutation(0, 99)

    except (TypeError, IndexError) as ex:
        print(f"[Execution Error] {ex}")
    else:
        print("\nFinal State Summary:", engine.get_summary())
```

Question 3 (20 Marks): Dictionaries and Sets

(a) [5 Marks] Theoretical Principles & Key Rules

- **Dictionaries (`{key: value}`)**: Ordered (Python 3.7+) collections storing key-value pairs.
  - **Key Rules**: Keys **must be unique and immutable** (strings, integers, floats, or tuples; lists can **never** be used as keys). Values can be any data type.
  - **Access**: `dict[key]` raises `KeyError` if the key is absent. `.get(key, default)` safely returns a default value without raising an exception.
  - **Merging**: `.update(dict2)` merges in-place. Python 3.9+ supports the merge operator `|`.
- **Sets (`{}`)**: Unordered collections of **unique** elements.
  - **Empty Set Gotcha**: Writing `s = {}` creates an **empty dictionary**. An empty set must be instantiated using `set()`.
  - **Removal**: `.remove(x)` raises a `KeyError` if `x` is missing. `.discard(x)` removes `x` safely **without** throwing an error if missing.
  - `frozenset`: An immutable, hashable set container.

(b) [5 Marks] Debugging & Syntax Check

Debugging Task: Locate and fix the 2 runtime/syntax errors:

```
# Task: Set initialization and dictionary assignment
data_set = {}
data_set.add(10)  # Error 1

invalid_dict = {: "Vector"}  # Error 2
```

- **Error 1**: `data_set = {}` creates a dictionary, causing `AttributeError: 'dict' object has no attribute 'add'`. Fix: Use `data_set = set()`.
- **Error 2**: `TypeError: unhashable type: 'list'`. Lists are mutable and cannot serve as dictionary keys. Fix: Use a tuple key `{(1, 2): "Vector"}`.

© [5 Marks] Simple Baseline Solution: Dictionary and Set Manager

```
# Baseline demonstration of dict and set operations
student_scores = {"Ravi": 85, "Alpha": 92}

# Safe Lookup & Insertion
print("Priya's Score:", student_scores.get("Priya", "Not Enrolled"))
student_scores.update({"Arun": 78, "Priya": 90})

# Set Duplicate Removal & Operations
raw_tags = ["python", "code", "python", "data"]
unique_tags = set(raw_tags)
unique_tags.add("script")
unique_tags.discard("missing_tag")  # Safe removal

print("Updated Dictionary:", student_scores)
print("Unique Tags Set:", unique_tags)
```

(d) [5 Marks] Complex Improvement: Modular OOP Student Database with State Validation and Custom Exception Handling

```
# --- Custom Exception Class ---
class StudentNotFoundError(Exception):
    """Raised when querying a non-existent student roll number."""
    def __init__(self, roll_no):
        self.roll_no = roll_no
        super().__init__(f"Student Record Error: Roll No '{roll_no}' not found in dat

# --- OOP Student Registry System ---
class StudentRegistrySystem:
    def __init__(self):
        # Dictionary storing roll_no -> {"name": str, "courses": set}
        self._records = {}
        self._all_offered_courses = {"Python", "DataStructures", "Mathematics", "Data
```

```
    def register_student(self, roll_no, name, course_list):
        """Registers student and validates course offerings using set algebra."""
        if not isinstance(roll_no, int) or roll_no <= 0:
            raise ValueError("Roll number must be a positive integer.")

        if roll_no in self._records:
            raise KeyError(f"Student Roll No {roll_no} is already registered.")

        # Convert course list to set for duplicate removal
        requested_courses = set(course_list)

        # Verify invalid courses using set difference
        invalid_courses = requested_courses - self._all_offered_courses
        if invalid_courses:
            raise ValueError(f"Cannot register: Unrecognized courses {invalid_courses

        self._records[roll_no] = {
            "name": name.strip(),
            "courses": requested_courses
        }

    def drop_course(self, roll_no, course_name):
        """Safely removes a course from student's enrolled set."""
        if roll_no not in self._records:
            raise StudentNotFoundError(roll_no)

        student_courses = self._records[roll_no]["courses"]
        if course_name not in student_courses:
            raise KeyError(f"Course '{course_name}' is not in student's current enro

        student_courses.discard(course_name)  # Safe set discard

    def get_student_summary(self, roll_no):
        """Safely retrieves student record dictionary."""
        record = self._records.get(roll_no)
        if not record:
            raise StudentNotFoundError(roll_no)
        return record

if __name__ == "__main__":
    registry = StudentRegistrySystem()
    try:
        registry.register_student(101, "Alpha", ["Python", "Mathematics", "Python"])
        registry.drop_course(101, "Mathematics")
```

```
        # Attempting invalid lookup
        registry.get_student_summary(999)

    except StudentNotFoundError as ex:
        print(f"[Lookup Error] {ex}")
    except (ValueError, KeyError) as ex:
        print(f"[Validation Error] {ex}")
    else:
        print("[Success] All registry operations executed cleanly.")
```

Question 4 (20 Marks): Input and Output (`input()` , `print()` , Formatting, File I/O)

(a) [5 Marks] Theoretical Core Concepts

- **Console Input (`input()`)**: Pauses program execution and reads user keyboard input, **always returning a string**. Numeric inputs must be converted explicitly using `int()` or `float()` .
- **Console Output (`print()`)**: Supports keyword parameters:
  - `sep` : Defines string placed *between* printed arguments (default: space).
  - `end` : Defines string printed at the *end* (default: newline `\n`).
- **String Formatting**: Modern f-strings `f"{var:.2f}"` allow inline variable interpolation and floating-point precision specification.
- **File I/O Modes**:
  - `'r'` : Read-only (default). File must exist.
  - `'w'` : Write-only. Creates file if missing; **overwrites existing content**.
  - `'a'` : Append. Adds content to the end without erasing.
  - `'x'` : Exclusive creation. Fails if the file already exists.
- **Context Manager (`with open(...) as f:`)**: Automatically manages file handle lifecycle and guarantees file closure even if exceptions occur.

(b) [5 Marks] Debugging & File Pointer Tracing

Tracing Task: Given a file `data.txt` containing `"Python\nProgramming"` :

```
with open("data.txt", "r") as f:
    line = f.readline()
    pos = f.tell()
    f.seek(0)
```

```
    full_text = f.read(3)
    print(f"Pos: {pos} | Text: {full_text}")
```

- **Output**: `Pos: 7 | Text: Pyt`
- *Explanation*: `readline()` reads `"Python\n"` (7 bytes including `\n`), moving the file pointer to byte offset `7`. `seek(0)` resets pointer back to start, and `read(3)` reads the first 3 characters `"Pyt"`.

© [5 Marks] Simple Baseline Solution: Student Mark Sheet File Writer/Reader

```
# Baseline file writing and reading script
filename = "marks_report.txt"

# Writing data using context manager
with open(filename, "w") as f:
    f.write("Alpha   85\n")
    f.write("Beta    92\n")

# Appending data
with open(filename, "a") as f:
    f.write("Gamma   78\n")

# Reading line by line
print("--- File Contents ---")
with open(filename, "r") as f:
    for line in f:
        name, score = line.strip().split()
        print(f"Student: {name:<10} | Score: {int(score):>3}")
```

(d) [5 Marks] Complex Improvement: Modular OOP File Logger & Analyzer Engine with Exception Safety

```
import os

class FileProcessingEngine:
    def __init__(self, filepath):
        self.filepath = filepath
```

```
    def write_records_safely(self, record_dict):
        """Writes dictionary records to disk with file mode safety."""
        if not isinstance(record_dict, dict):
            raise TypeError("Record data must be supplied as a dictionary.")

        try:
            # Mode 'w' overwrites, ensuring fresh report generation
            with open(self.filepath, "w", encoding="utf-8") as f:
                for name, score in record_dict.items():
                    f.write(f"{name},{score}\n")
            print(f"[I/O Success] Wrote {len(record_dict)} records to '{self.filepath

        except IOError as ex:
            raise IOError(f"File System Error writing to '{self.filepath}': {ex}")

    def read_and_parse_metrics(self):
        """Reads, parses, and calculates statistics from file with exception guards.'
        if not os.path.exists(self.filepath):
            raise FileNotFoundError(f"Target dataset file '{self.filepath}' does not

        scores = []
        try:
            with open(self.filepath, "r", encoding="utf-8") as f:
                for line_no, line in enumerate(f, start=1):
                    clean_line = line.strip()
                    if not clean_line:
                        continue  # Skip blank lines

                    parts = clean_line.split(",")
                    if len(parts) != 2:
                        print(f"[Format Warning] Line {line_no} malformed. Skipping.'
                        continue

                    name, score_str = parts
                    scores.append(float(score_str))

            if not scores:
                raise ValueError("Dataset file contains no valid numerical records.")

            mean_val = sum(scores) / len(scores)
            return len(scores), mean_val, min(scores), max(scores)

        except ValueError as ex:
            raise ValueError(f"Data parsing error in '{self.filepath}': {ex}")
        except IOError as ex:
            raise IOError(f"Read error encountered: {ex}")
```

```
if __name__ == "__main__":
    file_engine = FileProcessingEngine("student_dataset.csv")
    sample_students = {"Alpha": 88.5, "Beta": 92.0, "Gamma": 79.5}

    try:
        file_engine.write_records_safely(sample_students)
        count, avg, min_s, max_s = file_engine.read_and_parse_metrics()

        print("\n--- Dataset Metrics ---")
        print(f"Total Records : {count}")
        print(f"Average Score : {avg:.2f}")
        print(f"Score Range   : {min_s:.1f} to {max_s:.1f}")

    except (FileNotFoundError, IOError, ValueError, TypeError) as ex:
        print(f"[Engine Exception] {ex}")
```

Question 5 (20 Marks): Integrated Capstone — Student Records GUI Application (Tkinter + File I/O + Dictionaries + OOP + Custom Exceptions)

(a) [5 Marks] Desktop GUI Event-Driven Architecture

- **Tkinter Foundation**: Tkinter operates on an event-driven loop initiated by `window.mainloop()`.
- **Core UI Widgets**:
  - `Tk()` initializes root application window.
  - `Frame` acts as a sub-panel container for organizing layout.
  - `grid(row=r, column=c)` provides 2D tabular coordinate layout.
  - `Entry` collects single-line user string input.
  - `Listbox` displays scrollable sequence items.
  - `messagebox` triggers pop-up alerts (`showerror` , `showinfo` , `showwarning`).

(b) [10 Marks] & © [5 Marks] Complete Capstone Application Implementation

```
import tkinter as tk
from tkinter import messagebox
import os
```

```
# --- Custom Domain Exception ---
class InvalidGradeError(Exception):
    """Raised when an entered score is outside the 0 to 100 range."""
    def __init__(self, score):
        self.score = score
        super().__init__(f"Grade Boundary Error: Score {score:.1f} is invalid. Must b

# --- OOP Data Storage & Persistence Model ---
class StudentRecordModel:
    def __init__(self, storage_filename="students_db.txt"):
        self.storage_filename = storage_filename
        # Dictionary structure: {roll_no: {"name": str, "score": float}}
        self._student_dict = {}
        self.load_from_file()

    def add_student(self, roll_no, name, score):
        """Validates inputs and adds record to dictionary."""
        if roll_no in self._student_dict:
            raise KeyError(f"Student with Roll No {roll_no} is already registered.")

        if score < 0.0 or score > 100.0:
            raise InvalidGradeError(score)

        self._student_dict[roll_no] = {"name": name, "score": score}
        self.save_to_file()

    def get_all_records(self):
        return self._student_dict.items()

    def save_to_file(self):
        """Persists current dictionary records to disk."""
        with open(self.storage_filename, "w", encoding="utf-8") as f:
            for roll, data in self._student_dict.items():
                f.write(f"{roll},{data['name']},{data['score']}\n")

    def load_from_file(self):
        """Populates dictionary from disk file if available."""
        if not os.path.exists(self.storage_filename):
            return

        with open(self.storage_filename, "r", encoding="utf-8") as f:
            for line in f:
                parts = line.strip().split(",")
                if len(parts) == 3:
                    roll, name, score = parts
                    self._student_dict[int(roll)] = {"name": name, "score": float(sc
```

```
# --- Full Desktop Tkinter GUI Front-End ---
class StudentManagementGUI(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Student Records & Persistence Studio")
        self.geometry("520x420")
        self.config(bg="#f4f6f7")

        self.model = StudentRecordModel()
        self.build_ui()
        self.refresh_display()

    def build_ui(self):
        # 1. Header Banner Frame
        header = tk.Frame(self, bg="#2c3e50", pady=10)
        header.pack(side=tk.TOP, fill=tk.X)

        tk.Label(
            header,
            text="Student Record Management System",
            font=("Arial", 14, "bold"),
            fg="white",
            bg="#2c3e50"
        ).pack()

        # 2. Form Panel (LabelFrame with Grid Geometry)
        form_frame = tk.LabelFrame(
            self,
            text=" Add Student Record ",
            font=("Arial", 10, "bold"),
            bg="#f4f6f7",
            padx=10,
            pady=10
        )
        form_frame.pack(pady=10, fill=tk.X, padx=15)

        tk.Label(form_frame, text="Roll No:", bg="#f4f6f7").grid(row=0, column=0, st
        self.ent_roll = tk.Entry(form_frame, width=12)
        self.ent_roll.grid(row=0, column=1, pady=3, padx=5)

        tk.Label(form_frame, text="Name:", bg="#f4f6f7").grid(row=0, column=2, stick
        self.ent_name = tk.Entry(form_frame, width=15)
        self.ent_name.grid(row=0, column=3, pady=3, padx=5)

        tk.Label(form_frame, text="Score (0-100):", bg="#f4f6f7").grid(row=1, column
```

```
        self.ent_score = tk.Entry(form_frame, width=12)
        self.ent_score.grid(row=1, column=1, pady=3, padx=5)

        btn_add = tk.Button(
            form_frame,
            text="Save Student",
            command=self.handle_add_student,
            bg="#27ae60",
            fg="white",
            font=("Arial", 9, "bold")
        )
        btn_add.grid(row=0, column=4, rowspan=2, padx=10, sticky="nesw")

        # 3. Listbox Record Display Panel
        display_frame = tk.Frame(self, bg="#f4f6f7")
        display_frame.pack(pady=5, fill=tk.BOTH, expand=True, padx=15)

        self.scrollbar = tk.Scrollbar(display_frame, orient=tk.VERTICAL)
        self.record_listbox = tk.Listbox(
            display_frame,
            font=("Courier", 9),
            yscrollcommand=self.scrollbar.set
        )
        self.scrollbar.config(command=self.record_listbox.yview)

        self.record_listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        self.scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        # 4. Footer Summary Controls
        footer = tk.Frame(self, bg="#f4f6f7")
        footer.pack(side=tk.BOTTOM, fill=tk.X, pady=10, padx=15)

        tk.Button(
            footer,
            text="Compute Class Average",
            command=self.handle_calculate_average,
            bg="#2980b9",
            fg="white",
            font=("Arial", 9, "bold")
        ).pack(side=tk.LEFT)

    def handle_add_student(self):
        """Extracts input, validates formats, and handles custom exceptions."""
        try:
            roll_str = self.ent_roll.get().strip()
            name = self.ent_name.get().strip()
```

```
                score_str = self.ent_score.get().strip()

                if not roll_str or not name or not score_str:
                    raise ValueError("All fields are mandatory.")

                roll_no = int(roll_str)
                score = float(score_str)

                # Insert into backend model
                self.model.add_student(roll_no, name, score)

                # Clear entry inputs
                self.ent_roll.delete(0, tk.END)
                self.ent_name.delete(0, tk.END)
                self.ent_score.delete(0, tk.END)
                self.ent_roll.focus()

                self.refresh_display()
                messagebox.showinfo("Success", f"Student '{name}' added and saved to file

            except ValueError as ex:
                messagebox.showerror("Input Format Error", f"Validation Failed: {ex}")
            except KeyError as ex:
                messagebox.showwarning("Duplicate Entry", str(ex))
            except InvalidGradeError as ex:
                messagebox.showwarning("Grade Range Violation", str(ex))

        def refresh_display(self):
            """Reloads records from dictionary model into GUI Listbox."""
            self.record_listbox.delete(0, tk.END)
            for roll, data in self.model.get_all_records():
                formatted_line = f"Roll: {roll:<5} | Name: {data['name']:<12} | Score: {d
                self.record_listbox.insert(tk.END, formatted_line)

        def handle_calculate_average(self):
            """Calculates average score using dictionary values."""
            records = list(self.model.get_all_records())
            if not records:
                messagebox.showinfo("Class Summary", "No student records available.")
                return

            total = sum(data["score"] for _, data in records)
            avg = total / len(records)
            messagebox.showinfo("Class Metrics", f"Total Registered Students: {len(record

    if __name__ == "__main__":
```

```
app = StudentManagementGUI()
app.mainloop()
```